

ocean Ai

// session 09 final voyage

Polish & showcase

Choose one meaningful extension. Finish the game.
Show what you built.



You replaced the engine one idea at a time.

01

Variables and setup

02

Player decisions

03

Your planet list

04

Your game loop

05

Planet dictionaries

06

Focused functions

07

The Ship class

08

Encounters and morale

Make one choice. Finish it well.

Plan → build → test → clean → commit → present.

40 minutes

Build one extension that fits the time.

Final commit

Leave the repository working and readable.

2 minutes

Show the galaxy, one proud code choice,
and one live round.

What would make your game feel finished?

New encounter

Nebula, station, anomaly or distress signal.

Better scanner

Water, morale effect and a recommendation.

Difficulty

Easy, normal and hard resource rules.

ASCII art

Add a distinctive title, victory or defeat moment.

Statistics

Track stops, skips, damage or final morale.

Narrative

Let current morale change the story text.



// plan first protect the finish

Can you describe the smallest version in one sentence?

what changes

which files

how I will test it

One constant can choose a rule.

```
DIFFICULTY = "normal" # "easy", "normal", or "hard"
```

```
def oxygen_cost(difficulty):  
    if difficulty == "easy":  
        return 5  
    elif difficulty == "hard":  
        return 11  
    else:  
        return 8
```

uses prior knowledge

A variable selects a branch. A pure function hands back a fixed jump cost.

Test all three settings before connecting it to the loop.

Turn existing data into useful advice.

```
def scanner_message(destination, morale):  
    danger = destination["danger_level"]  
    water = destination["has_water"]  
  
    if danger ≥ 4:  
        advice = "recommend bypassing"  
    else:  
        advice = "recommend stopping"  
  
    water_note = "water detected" if water else "no water detected"  
    return f"{advice} | {water_note} | morale {morale}"
```

dictionary + conditionals

Read danger and water. Choose one recommendation. Return one clear scanner line.

A dictionary can remember the whole run.

```
def new_statistics():  
    return {"visited": 0, "skipped": 0, "damage": 0}  
  
def record_stop(statistics, damage):  
    statistics["visited"] = statistics["visited"] + 1  
    statistics["damage"] = statistics["damage"] + damage  
  
def record_skip(statistics):  
    statistics["skipped"] = statistics["skipped"] + 1
```

dictionary + functions

Start counters at zero. Update only when the related event happens. Print them after victory or defeat.

The same event can tell a different story.

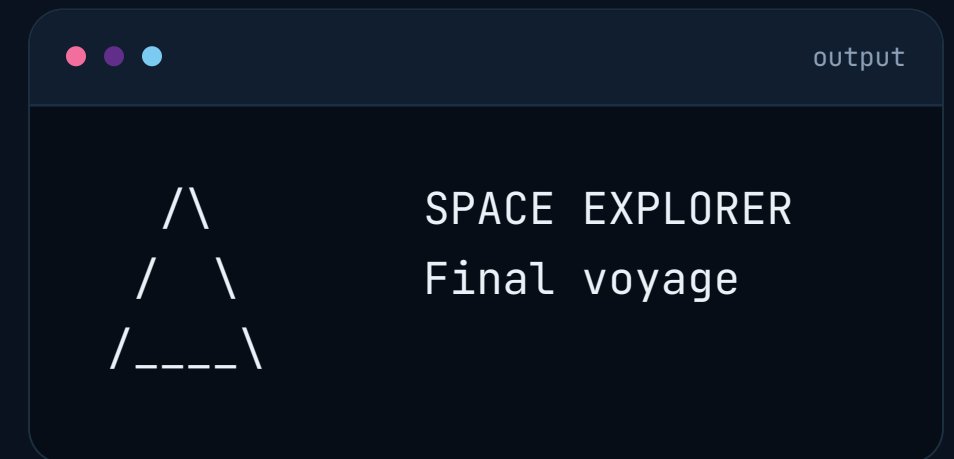
```
def raider_message(morale):  
    if morale ≥ 110:  
        return "The crew holds formation and drives the raiders  
away."  
    elif morale ≤ 80:  
        return "The shaken crew struggles to defend the ship."  
    else:  
        return "The crew fights off the raiders."
```

morale + conditionals

The mechanics stay fixed. The language reflects how the crew is doing.

Polish can be simple and visible.

```
def show_title():  
    print("  /\\"      SPACE EXPLORER")  
    print(" /  \\"      Final voyage")  
    print("/____\\")
```



Build the smallest version first.

- 01 Write the success condition.
- 02 Change one thing.
- 03 Run the game immediately.
- 04 Keep or undo based on evidence.
- 05 Stop adding features with 10 minutes left.

Success looks like

One completed extension, a winnable and losable game, and enough time to clean and commit.

Try to prove your feature wrong.

expected path

Use the feature exactly as designed.

other branch

Try the opposite choice or boundary.

whole game

Reach both victory and defeat.

```
$ uv run game.py
```

Leave code your future self can read.

✓ Every function does one job.

✓ No dead or commented-out code.

✓ Constants reflect your final design.

✓ Variable names explain their values.

✓ The game is winnable and losable.

✓ The last run completed without errors.

Your ninth checkpoint.

Run first. Save everything. Confirm the push.

```
$ uv run game.py
$ git status
$ git add -A
$ git commit -m "session 9: final polish"
$ git push
```

The repository tells the story.

`constants.py`

Your ship and galaxy

`game.py`

Your loop, functions and encounters

`player.py`

Your Ship class

`history`

Nine sessions of progress

The final commit is visible on GitHub

Show the game. Show the thinking.

1. Your galaxy

Name one world and why you designed it that way.

2. Your code

Point to one choice or structure you are proud of.

3. One round

Play live and narrate one decision.

You can point to evidence.

Set up and run a Python project.

Read code you did not write.

Control decisions and repeated work.

Model collections and structured data.

Write focused functions and classes.

Use Git to save and share progress.

Python can take you in many directions.

Web

Build services and sites.

Data

Explore evidence and patterns.

Games

Add graphics, sound and interaction.

Automation

Remove repetitive work.

AI

Train models after the foundations.

Your idea

Keep the repository growing.

// zero → one complete

You wrote real code.
You built this.

Nine sessions. One finished project. A GitHub history that proves the journey.

